

Section 1: Introduction & Features

1. Feature extraction transforms raw visual data into:

- A. Larger raw data
- B. More compact and meaningful representation ☒
- C. Audio signals
- D. Only color channels

2. Raw visual data is often too ____ for direct analysis.

- A. Simple
- B. High-dimensional and complex ☒
- C. Low-dimensional
- D. Binary

3. Features should capture:

- A. All irrelevant details
- B. Essential information about objects/patterns ☒
- C. Only noise
- D. Only pixel coordinates

4. Local features are extracted from:

- A. Entire image
- B. Specific regions such as keypoints, corners, patches ☒
- C. Only color histograms
- D. Only edges

5. Global features are computed:

- A. Only on corners
- B. Over the entire image ☒
- C. Only on patches
- D. Only on keypoints

6. Which is an example of global feature?

- A. Keypoint
- B. Corner

C. Color histogram ☒

D. Small patch

7. Histogram-based methods capture:

A. Only shape

B. Statistical information like color or texture distributions ☒

C. Only depth

D. Only motion

8. Gabor filters and Haar wavelets are examples of:

A. Histogram methods

B. Filter-based methods ☒

C. Deep learning methods

D. Color spaces

9. CNNs extract features:

A. Manually

B. Directly from data through layers ☒

C. Only by histograms

D. Only by Sobel

10. Which is an application of feature extraction?

A. Face recognition ☒

B. File compression only

C. Audio recording

D. Printing

11. Feature extraction challenges include:

A. Lighting, scale, orientation, noise ☒

B. Only color

C. Only resolution

D. No challenges

12. Choosing appropriate feature extraction techniques and parameters is:

A. Not important

B. Critical for successful computer vision applications ☒

- C. Only for audio
 - D. Only for video games
13. Extracted features are often used with:
- A. Matching or learning algorithms ☒
 - B. Only printers
 - C. Only monitors
 - D. Only keyboards
14. Local features are often used for:
- A. Image matching and object detection ☒
 - B. Color printing
 - C. Audio synthesis
 - D. File storage
15. Image moments are an example of:
- A. Local feature
 - B. Global feature ☒
 - C. Edge detector
 - D. Filter
-

Section 2: HOG

16. HOG stands for:
- A. Histogram of Optical Gradients
 - B. Histogram of Oriented Gradients ☒
 - C. High Order Gradients
 - D. Histogram of Oriented Geometry
17. HOG is particularly useful for:
- A. Audio detection
 - B. Object detection and pedestrian detection ☒
 - C. Color printing
 - D. File compression

18. HOG captures information about:

- A. Local gradient or edge patterns ✓
- B. Only color
- C. Only depth
- D. Only motion

19. In HOG, image is divided into:

- A. Small cells ✓
- B. Large blocks only
- C. Audio frames
- D. Color bins

20. For HOG, gradient computation typically uses:

- A. Sobel operator ✓
- B. Fourier transform
- C. K-means
- D. PCA

21. HOG first converts input image to:

- A. RGB
- B. Grayscale ✓
- C. HSV
- D. Binary only

22. In HOG, histogram bins represent:

- A. Color ranges
- B. Different orientation ranges ✓
- C. Pixel coordinates
- D. Depth values

23. Block normalization in HOG reduces effects of:

- A. Lighting variations ✓
- B. Motion
- C. Audio noise
- D. Compression

24. In HOG, neighboring cells are grouped into:

- A. Bins
- B. Blocks ☒
- C. Layers
- D. Channels

25. Final HOG feature vector is formed by:

- A. Averaging all pixels
- B. Concatenating normalized histograms from blocks ☒
- C. Sorting colors
- D. Removing edges

26. Typical cell size example in HOG manual:

- A. 8x8 pixels ☒
- B. 16x16 pixels
- C. 32x32 pixels
- D. 64x64 pixels

27. Typical block example in HOG manual:

- A. 2x2 cells ☒
- B. 3x3 cells
- C. 4x4 cells
- D. 5x5 cells

28. HOG code uses function:

- A. cv2.hog
- B. skimage.feature.hog ☒
- C. cv2.HOGDescriptor
- D. numpy.hog

29. In HOG code, pixels_per_cell=(8,8) means:

- A. Block size
- B. Cell size ☒
- C. Image size
- D. Bin count

30. In HOG code, cells_per_block=(2,2) means:

- A. Cell size
 - B. Number of cells per block ☒
 - C. Number of orientations
 - D. Number of channels
-

Section 3: LBP

31. LBP stands for:

- A. Local Binary Pixel
- B. Local Binary Pattern ☒
- C. Linear Binary Pattern
- D. Local Basic Pattern

32. LBP is commonly used for:

- A. Texture classification and face recognition ☒
- B. Audio compression
- C. Motion detection only
- D. Color printing

33. LBP operates on:

- A. RGB images only
- B. Grayscale images ☒
- C. Depth images only
- D. Audio signals

34. In LBP, each pixel is compared with:

- A. Its neighboring pixels ☒
- B. Whole image
- C. Color histogram
- D. Sobel kernel

35. In LBP, if neighbor intensity is greater or equal to center, assign:

- A. 0
- B. 1 ☒

- C. -1
- D. 255

36. In LBP, if neighbor intensity is less than center, assign:

- A. 0 ☒
- B. 1
- C. -1
- D. 255

37. LBP binary values are concatenated to form:

- A. Color histogram
- B. LBP pattern ☒
- C. Sobel gradient
- D. HOG block

38. The LBP histogram serves as:

- A. Feature vector ☒
- B. Kernel
- C. Threshold
- D. Edge map

39. LBP code uses function:

- A. cv2.LBP
- B. skimage.feature.local_binary_pattern ☒
- C. np.lbp
- D. cv2.local_binary_pattern

40. In LBP code, method='uniform' is used to:

- A. Reduce number of patterns / use uniform patterns ☒
- B. Increase radius
- C. Convert to RGB
- D. Detect edges

41. In LBP code, radius = 1 defines:

- A. Cell size
- B. Radius of circular neighborhood ☒

C. Number of bins

D. Threshold

42. In LBP code, $n_points = 8 * radius$ defines:

A. Number of neighboring pixels to consider ✓

B. Image width

C. Histogram bins

D. Kernel size

43. LBP is particularly useful for:

A. Texture classification ✓

B. Pedestrian detection only

C. Audio recognition

D. Depth estimation

44. LBP captures local patterns by:

A. Comparing intensity of a pixel with its neighbors ✓

B. Computing Fourier transform

C. Applying Canny

D. Color quantization

45. LBP histogram counts occurrences of:

A. Colors

B. Each LBP pattern ✓

C. Gradients

D. Edges

Section 4: Histogram of Color

46. Histogram of Color captures:

A. Distribution of color intensities ✓

B. Edge directions

C. Texture energy

D. Gradient magnitude

47. Histogram of Color divides color space into:

- A. Bins ☒
- B. Blocks
- C. Cells
- D. Layers

48. Common color spaces for Histogram of Color:

- A. RGB or HSV ☒
- B. Only grayscale
- C. Only binary
- D. Only CMYK

49. Histogram of Color counts:

- A. Number of pixels in each bin ☒
- B. Number of edges
- C. Number of corners
- D. Number of gradients

50. Histogram of Color is an example of:

- A. Local feature
 - B. Global feature ☒
 - C. Edge detector
 - D. Filter
-

Section 5: HED

51. HED stands for:

- A. Histogram of Edge Detection
- B. Histogram of Edge Directions ☒
- C. High Edge Density
- D. Histogram of Edge Distribution

52. HED captures:

- A. Distribution of edge orientations ☒
- B. Color distribution

C. Texture energy

D. Pixel coordinates

53. HED is useful for:

A. Texture analysis, object recognition, image segmentation ✓

B. Audio processing

C. File compression

D. Printing

54. In HED, edge detection can use:

A. Only Canny

B. Canny, Sobel, or Scharr ✓

C. Only K-means

D. Only PCA

55. In HED, orientation at each pixel is calculated using:

A. Arctangent function ✓

B. Sine function

C. Cosine function

D. Logarithm

56. In HED, 360 degrees can be divided into:

A. 4 bins

B. 8 bins for octagonal directions ✓

C. 16 bins

D. 32 bins

57. HED feature vector is:

A. Histogram of edge directions ✓

B. Color histogram

C. HOG vector

D. LBP histogram

58. In HED code, gradient orientation is computed by:

A. $\text{np.arctan2}(\text{gradient_x}, \text{gradient_y}) * 180 / \text{np.pi}$

B. $\text{np.arctan2}(\text{gradient_y}, \text{gradient_x}) * 180 / \text{np.pi}$ ✓

C. `np.tan(gradient_y/gradient_x)`

D. `np.sin(gradient_x)`

59. HED histogram bins in code:

A. 4

B. 8 ☒

C. 16

D. 256

60. HED histogram range in code:

A. (0, 180)

B. (0, 360) ☒

C. (0, 90)

D. (0, 255)

Section 6: HIG

61. HIG stands for:

A. Histogram of Image Gradients

B. Histogram of Intensity Gradients ☒

C. High Intensity Gradient

D. Histogram of Intensity Groups

62. HIG captures:

A. Distribution of intensity gradients ☒

B. Color distribution

C. Edge directions only

D. Texture energy

63. HIG is useful for:

A. Object recognition, texture analysis, image classification ☒

B. Audio processing

C. Printing

D. File compression

64. Gradient magnitude formula:

- A. $G = dx + dy$
- B. $G = \sqrt{dx^2 + dy^2}$ ✓
- C. $G = dx^2 - dy^2$
- D. $G = dx * dy$

65. Gradient orientation formula:

- A. $\Theta = \text{atan2}(dx, dy)$
- B. $\Theta = \text{atan2}(dy, dx)$ ✓
- C. $\Theta = \sin(dx/dy)$
- D. $\Theta = \cos(dy/dx)$

66. In HIG, gradient can be computed using:

- A. Sobel, Scharr, or other gradient methods ✓
- B. Only Canny
- C. Only K-means
- D. Only PCA

67. In HIG, histogram range for orientations is typically:

- A. 0 to 180 degrees
- B. 0 to 360 degrees ✓
- C. 0 to 90 degrees
- D. 0 to 255 degrees

68. HIG histogram can be normalized to make it:

- A. Scale-invariant ✓
- B. Colorful
- C. Binary
- D. Noisy

69. HIG feature vector is:

- A. Histogram of gradient orientations ✓
- B. Color histogram
- C. LBP histogram
- D. HOG block

70. HIG is similar to HOG but usually:

- A. Global instead of local cells/blocks ✓
 - B. Only for audio
 - C. Only for video
 - D. Only for depth
-

Section 7: Texture Energy & Contrast

71. Texture energy is computed by:

- A. Sum of pixel values
- B. Sum of squared pixel values in neighborhood ✓
- C. Average of pixel values
- D. Product of pixel values

72. Texture contrast is computed as:

- A. Mean of pixel values
- B. Standard deviation of pixel values in neighborhood ✓
- C. Sum of squared pixels
- D. Maximum pixel value

73. Texture contrast formula in manual:

- A. $C = \text{var}(\text{pixel values})$
- B. $C = \sqrt{\text{var}(\text{pixel values})}$ ✓
- C. $C = \text{sum}(\text{pixel values})$
- D. $C = \text{max}(\text{pixel values})$

74. Common neighborhood sizes for texture analysis:

- A. 3x3 or 5x5 ✓
- B. 8x8 or 16x16
- C. 32x32 or 64x64
- D. 1x1 or 2x2

75. Texture energy histogram captures:

- A. Distribution of texture energy values ✓
- B. Distribution of colors

C. Distribution of edges

D. Distribution of gradients

76. Texture contrast histogram captures:

A. Distribution of contrast values ☒

B. Distribution of colors

C. Distribution of orientations

D. Distribution of gradients

77. Texture energy and contrast features can be used for:

A. Texture classification, segmentation ☒

B. Audio processing

C. File compression

D. Printing

78. In texture code, energy uses `cv2.filter2D(image**2, -1, np.ones((3,3)))` because:

A. It sums squared pixel values in neighborhood ☒

B. It blurs image

C. It detects edges

D. It converts to grayscale

79. In texture code, histograms are created using:

A. `np.histogram` ☒

B. `cv2.histogram`

C. `plt.hist`

D. `skimage.histogram`

80. Normalizing texture histograms is:

A. Required

B. Optional ☒

C. Impossible

D. Only for color

Section 8: Filtering & Convolution

81. Filtering involves applying a ____ to an image.

A. Small matrix called kernel/filter ☒

B. Large database

C. Color space

D. Histogram

82. Each element of the filter represents:

A. Weighted contribution to new pixel value ☒

B. Pixel coordinate

C. Color channel

D. Edge direction

83. Convolution involves:

A. Sliding filter over image, multiplying, summing

B. Sliding filter over image, element-wise multiplying, summing ☒

C. Sorting pixels

D. Averaging colors

84. Convolution result is called:

A. Output or convolved image ☒

B. Histogram

C. Feature vector

D. Kernel

85. Box blur replaces each pixel with:

A. Average of neighboring pixels ☒

B. Maximum of neighbors

C. Minimum of neighbors

D. Median of neighbors

86. Box blur kernel in manual:

A. `np.ones((3,3))`

B. `np.ones((3,3), dtype=np.float32) / 9` ☒

C. `np.eye(3)`

D. `np.zeros((3,3))`

87. Gaussian blur uses:

- A. Gaussian-shaped kernel ☒
- B. Box kernel
- C. Sobel kernel
- D. Emboss kernel

88. Gaussian blur provides smoother results compared to:

- A. Box blur ☒
- B. Sobel
- C. Scharr
- D. Canny

89. Sobel and Scharr operators detect:

- A. Edges by emphasizing rapid changes in pixel values ☒
- B. Colors
- C. Textures only
- D. Corners only

90. Embossing creates:

- A. 3D effect by emphasizing differences in neighboring pixel values ☒
- B. Blur
- C. Edge map
- D. Histogram

91. Standard convolution is also known as:

- A. Full convolution ☒
- B. Valid convolution
- C. Same convolution
- D. Strided convolution

92. Valid convolution (no padding) produces output image:

- A. Same size as input
- B. Smaller than input ☒
- C. Larger than input
- D. Infinite

93. Same convolution adds:

- A. Zero-padding ☒
- B. Random noise
- C. Blur
- D. Color

94. Same convolution ensures output image has:

- A. Same dimensions as input ☒
- B. Smaller dimensions
- C. Larger dimensions
- D. No dimensions

95. Valid convolution with strides:

- A. Skips some pixels based on specified stride ☒
 - B. Adds padding
 - C. Keeps same size
 - D. Blurs image
-

Section 9: Edge Detection & Canny

96. Edge detection identifies:

- A. Boundaries within an image ☒
- B. Colors
- C. Textures only
- D. Histograms

97. Edges correspond to changes in:

- A. Color, intensity, or texture ☒
- B. Only color
- C. Only intensity
- D. Only texture

98. Gradient-based edge detection uses:

- A. Gradient of pixel intensities ☒
- B. Histogram

C. Color space

D. Fourier transform

99. Common gradient-based operators:

A. Sobel, Canny, LBP

B. Sobel, Prewitt, Scharr ☒

C. HOG, LBP, HED

D. K-means, PCA, SVM

100. Canny edge detector was developed by:

A. John F. Canny ☒

B. David Marr

C. Sobel

D. Prewitt

101. Canny edge detector was developed in:

A. 1976

B. 1986 ☒

C. 1996

D. 2006

102. First step in Canny:

A. Gaussian smoothing ☒

B. Non-maximum suppression

C. Hysteresis

D. Gradient calculation

103. Gaussian smoothing in Canny reduces:

A. Noise ☒

B. Edges

C. Color

D. Resolution

104. Second step in Canny:

A. Gaussian smoothing

B. Gradient calculation ☒

C. Non-maximum suppression

D. Hysteresis

105. Canny gradient calculation uses:

A. Sobel operators ☒

B. LBP

C. HOG

D. K-means

106. Non-maximum suppression in Canny:

A. Thins edges by keeping local maxima ☒

B. Blurs edges

C. Adds noise

D. Colors edges

107. Hysteresis in Canny uses:

A. One threshold

B. Two thresholds ☒

C. Three thresholds

D. No thresholds

108. In Canny, pixels above high threshold are:

A. Strong edge points ☒

B. Weak edge points

C. Non-edge points

D. Noise

109. In Canny, pixels between low and high threshold are:

A. Strong edge points

B. Weak edge points ☒

C. Non-edge points

D. Noise

110. In Canny, weak edge points are kept if:

A. Connected to strong edge points ☒

B. Isolated

C. Above high threshold

D. Below low threshold

111. Canny parameter sigma controls:

A. Gaussian filter kernel size / smoothing ☒

B. Threshold

C. Gradient direction

D. Color space

112. Larger sigma in Canny results in:

A. Smoother image but loss of fine details ☒

B. Sharper edges

C. More noise

D. No edges

113. Canny threshold1 in OpenCV is:

A. Low threshold ☒

B. High threshold

C. Sigma

D. Aperture size

114. Canny threshold2 in OpenCV is:

A. Low threshold

B. High threshold ☒

C. Sigma

D. Aperture size

115. Canny code uses function:

A. `cv2.canny(image, 100, 200)`

B. `cv2.Canny(image, threshold1=100, threshold2=200)` ☒

C. `cv2.edge(image, 100, 200)`

D. `cv2.CannyEdge(image, 100, 200)`

Section 10: Common Edge Techniques

116. LoG stands for:

- A. Laplacian of Gaussian ✓
- B. Logarithm of Gradient
- C. Local of Gradient
- D. Laplacian of Gradient

117. LoG combines:

- A. Gaussian smoothing and Laplacian edge detection ✓
- B. Sobel and Prewitt
- C. HOG and LBP
- D. Canny and Hysteresis

118. Prewitt operator uses:

- A. 3x3 kernels for horizontal and vertical gradients ✓
- B. 5x5 kernels
- C. 7x7 kernels
- D. 1x1 kernels

119. Marr-Hildreth edge detector uses:

- A. LoG after Gaussian smoothing ✓
- B. Sobel only
- C. Prewitt only
- D. Canny only

120. Sobel-Feldman operator emphasizes:

- A. Diagonal edges in addition to horizontal and vertical edges ✓
- B. Only horizontal edges
- C. Only vertical edges
- D. Only color edges

121. CNN-based edge detection can:

- A. Learn complex edge patterns and adapt to domains ✓
- B. Only detect straight lines
- C. Only detect circles
- D. Only detect colors

122. Edge detection reduces data while:

- A. Highlighting essential features ☒
- B. Removing all features
- C. Adding noise
- D. Changing colors

123. Gradient magnitude high indicates:

- A. Edge likely present ☒
- B. No edge
- C. Smooth region
- D. Color change only

124. Scharr operator is:

- A. A gradient-based edge detector ☒
- B. Not gradient-based
- C. Only for color
- D. Only for texture

125. Canny is known for:

- A. Accuracy and noise reduction ☒
- B. Speed only
- C. Color detection
- D. Texture only